Kombi Shift 2 final project report

Project identity and submission status

HAI311 Game Development, University of Zimbabwe. Group 6. Prepared 8 November 2026 for the 13 November deadline. This report describes the current KombiShift2 Unity project, not the earlier UnityStarter/UnityHDRP prototypes. Human tester observations, personal contribution verification, course-book editions and source redistribution permissions still require the team's review. Those items are not represented as completed.

Members: Leroy Mapunzwana R2420278; Jayden Mapasure R2420211; Tadiwanashe Nzenze R2420233; Euclide Mtisi R2420207; Tendai Mapiko R2420229.

The game is a 3D arcade passenger-driving game on a Harare-inspired route. The player drives a Hiace, collects waiting passengers before two rival kombis, and delivers the stage target to Mbare before time expires. The design prioritises a complete reliable shift, readable navigation and observable competition.

Audience and user personas

Intended audience: casual keyboard/mouse players aged 14 and above, particularly students familiar with Zimbabwean public transport. This is a design hypothesis, not a demographic research result. Vehicle handling is deliberately upright and simplified rather than a professional driver-training simulation.

Persona A: a 16-year-old occasional player who recognises kombi culture, wants a short entertaining shift and needs clear pickup/terminal cues. Design response: visible stop markers, a route minimap, automatic boarding and a forgiving introductory stage.

Persona B: a 21-year-old student seeking score improvement on a modest laptop. Design response: a harder second stage, persistent best earnings, adjustable camera distance, disabled cast shadows and controlled minimap refresh.

Persona C: a 30-year-old new player who needs explicit controls and predictable goals. Design response: How to Play, keyboard menu focus, visible speed/time/target, pause/resume, master volume and camera sensitivity settings. Real accessibility/usability testing is still required. In-game passenger personas are character data, distinct from these player personas.

Genre mechanics and core loop

Genre: arcade driving with timed passenger competition. Core loop: inspect stage target, locate a stop, drive/brake, wait for boarding, advance along the route, deliver at Mbare, read the result and replay or progress.

Driving uses W/S/A/D or arrows. Space applies the parking brake and cuts throttle. Boarding starts automatically below 2 km/h within the active station's service radius, with a free seat. Holding Space is the practical way to remain stopped; the service condition itself is speed/distance based. Driving away cancels unfinished boarding and people return to their station. A station reservation prevents player/rival double collection.

The player earns $2 for each passenger who actually reaches a seat. On success, a time bonus of round(remaining seconds * 0.08) is added. This is intentionally transparent: boarding income and success bonus are separate rules. Best successful earnings persist per stage. The current game does not claim fuel purchases, repairs, passenger patience pricing, individual destination fares or the older concept's conductor economy.

Two playable stages and initial balance

Both stages reuse the existing city scene and route. A stage is a different set of gameplay rules; a separate map is not required by the brief. Change the stage with the title-screen selector or F2. A successful Morning Shift offers Next: Rush Hour.

Morning Shift: 600 seconds; deliver at least one passenger; rivals travel at 6 metres/second and wait 45 seconds plus a 6-second stagger. It provides time to learn braking, boarding and navigation.

Rush Hour: 420 seconds; deliver at least six passengers; rivals travel at 9 metres/second and wait only 8 seconds plus the stagger. The player has less time, more aggressive competition and a higher delivery target. Geography, controls and fare stay consistent so the challenge change is understandable.

These are initial balancing decisions, not conclusions from authentic testers. Test completion rate, first-stop time, number collected, rival losses and perceived fairness. If novices repeatedly lose every queue, increase the head start or reduce rival speed. Record the actual change and retest both stages. Do not tune only for an automated staged test.

Start to finish journey and annotated storyboard



Figure 1. Final current-version WebGL title with stage target and selection, captured during browser verification on 8 November. This is technical evidence, not an authentic human tester session.

Panel 1 - Title: live Hiace/city background, Start Shift, How to Play and Settings. The stage selector states minutes, delivery target and best score. Annotation: the player should know the goal before starting.

Panel 2 - Instructions: controls, stopping/boarding and terminal goal. Annotation: automatic boarding removes an extra interaction key; the stop speed must be visible.

Panel 3 - Approach: minimap marks player, next stop and rivals; the timer and target remain visible. Annotation: the route is ordered, and stops emptied by rivals can be skipped.

Panel 4 - Service: seated passengers stand, walk together through the doorway and occupy distinct seats. Annotation: income is credited after seating, not when a boarding animation merely begins.

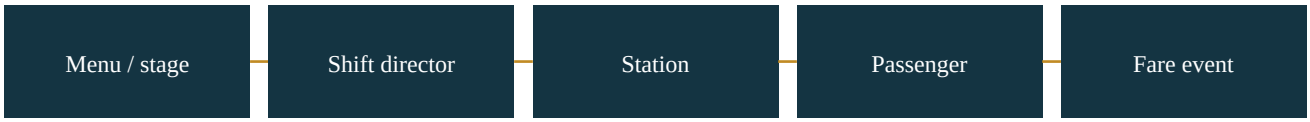
Panel 5 - Delivery: after pickup, follow the next stop or terminal. A full cabin routes to Mbare automatically. Annotation: boarding another station must not block a full vehicle from finishing.

Panel 6 - Results: wait for delivered passengers to reach terminal seats, freeze the world, display outcome/earnings/best score, then replay, return or advance to Rush Hour. Annotation: empty/insufficient delivery and timeout are failures.

UI flow: Title -> Help/Settings -> Title -> selected stage -> Pause -> Resume, Restart or Menu. Gameplay -> Success/Failure -> Replay/Menu. Morning success -> Rush Hour. Exit is available on title.

The earlier low-fidelity HTML prototype is retained as historical material. Its earlier concepts are not all current features. No invented early feedback or backdated storyboard history is claimed. The current storyboard documents the final implementation and design decisions.

Unity architecture and script communication



Director owns time/outcomes. Station owns reservations. Passenger owns movement.

Seat arrival -> callback -> director income -> HUD/result. Input -> RCC physics.

The configured build scenes are KombiMainMenu and KombiShift_FirstShift. The playable hierarchy groups systems/lighting, environment, roads/route, player/camera, stations and traffic/rivals. A reusable Hiace_Player prefab stores the vehicle setup; passenger prefabs reuse six character appearances. Assets/KombiShift2 holds project-specific code/art; vendor folders retain paths expected by their tools.

KombiMainMenu saves settings, selects the challenge and asynchronously opens the gameplay scene. KombiChallenge defines each stage's duration, target and rival settings, and persists best earnings. KombiShiftGameDirector owns timer, stop progression, service, fares and outcomes. It publishes OnFareEarned, OnPauseChanged and OnShiftFinished, allowing listeners to react without duplicating the rules.

PassengerStation owns queue reservation and seat allocation. It starts boarding coroutines and calls back only when a passenger has actually boarded. StationPassenger owns states Seated, Walking, Onboard, Returning, Alighting and Delivered. The director retains the onboard manifest and waits for terminal delivery before showing success.

RccHiaceTestBootstrap converts keyboard input into public RCC vehicle inputs and configures KombiPowertrain after vehicle registration. Its historical name is not an assertion that normal gameplay is only a test. The old autonomous batch smoke behaviour now requires the explicit --kombi-smoke switch. KombiVehicleStability supplies upright/grounding helpers. KombiDriveCamera owns chase/driver/orbit behaviour; KombiNavigationHud draws route/status. KombiWeatherController remains a weather support system, not a separate claimed highway level.

Physics animation and feedback



Figure 2. Earlier native grounding verification of the current vehicle/city, retained as historical technical evidence.

The player uses the imported RCC controller with a dynamic Rigidbody and WheelColliders. The team's integration configures input, gearing, speed limiting and stability; it does not claim original authorship of the RCC engine. Ground/camera/obstacle raycasts support contact, visibility and navigation. All-wheel parking braking cuts throttle; wheel mounts are protected from deformation/detachment. The governor gradually reduces throttle between 95 and 105 km/h, with a separate emergency stability ceiling.

StationPassenger uses a valid imported humanoid rig and procedural pose blending to move between seated and walking behaviour. Animator is used for rig setup and then procedural code poses the skeleton. We do not claim downloaded sitting/walking clips or a conventional Animator state graph. The observable transitions are the animation implementation; confirm any lecturer-specific Animator expectation.

Visible feedback includes walking/seating, route and rival markers, service notices, fares/time and results. Audio includes existing RCC engine layers plus generated menu focus/select sounds. The title and pause screen expose master volume. Camera sensitivity/distance are adjustable. Full device audibility and novice usability are human tests, not inferred from configuration.

Autonomous rival decision logic



Figure 3. Saved current-project passenger/rival service capture. This is automated staging, not an authentic external tester session.

Each rival has Cruise, ChasePassenger, PullIn, Boarding and Depart states. It searches stations with waiting, unreserved passengers, chooses the nearest route-index target, travels through waypoints, pulls into the service point and boards via the same station reservation mechanism. Obstacle sphere casts make it wait behind vehicles instead of teleporting through them.

No target produces actual route cruising; a full cabin produces actual departure movement along the route. Finished shifts stop rivals. There is no current Block/Flee behaviour. Waypoint navigation is deliberately simpler than a general road-network pathfinder. Extended blocking and all-queues-empty cases remain important manual fairness tests.

Runtime optimisation and performance method

The current traffic code replaces repeated allocating SphereCastAll/RaycastAll queries with reusable hit arrays. If a buffer is full, the runtime falls back to the complete query, preserving collision coverage instead of silently discarding obstacles. Camera/rival/stability code also uses buffered casts. The minimap refreshes every 0.15 seconds and skips work while paused/finished. Shadows are disabled by deliberate presentation settings.

The current measurement fixture opens the actual city scene, uses its traffic vehicles, warms both query modes, and samples 120 times per mode with 20 repeats per sample. It alternates mode order and verifies identical hit counts. Before each sample it forces collection, then records Mono used-heap delta and query elapsed time to CSV. This is controlled editor CPU-query evidence, not a GPU benchmark, whole-game FPS claim or human playtest. The runtime's GetAllocatedBytesForCurrentThread returned zero in both modes, so that unusable counter was replaced rather than reported as an improvement.

Read Docs/Validation/Assessment/traffic-query-results.json and traffic-query-samples.csv for actual figures and scope. The editor reported a shutdown signal after one profiling run; results written before shutdown are retained with that limitation. Repeat on the weakest team laptop and record normal gameplay frame-time evidence before making hardware claims.

Verification and human playtesting

Automated integration checks stage the actual player at stops, isolate rivals for service/outcome cases, observe six physical seat arrivals, fare events, pause freeze, stage tuning, successful delivery, clean replay, empty delivery, timeout and return to menu. These are opt-in tests and do not run in an ordinary launch. Test reports record Unity, platform, stage, time and scope. Tests with staged positions are not represented as a human-driven full shift.

Existing earlier current-project suites cover grounding, passenger service, menu/camera and powertrain. They remain separately dated evidence. New results must not be combined into a claim that every historical test was rerun together. Current test/build results are listed in the evidence manifest.

At least three authentic testers must complete tasks on a recorded build/device. Have each find the stage selector, explain the goal, drive/stop, board, navigate, pause/change audio, complete or fail, and compare stages. Record consent, anonymous participant ID, date/build/device, observed outcome, severity and comments. Collect no unnecessary personal information in public files.

Prioritise at least three real findings, implement fixes, record affected file/commit, and repeat the same task with a tester. The prepared CSVs are empty by design; an automated fix is not fabricated human-feedback evidence. Group members should personally play both stages even before external sessions.

Packaging deployment and compatibility

Unity version is 6000.6.2f1. The current project uses the Built-In pipeline. Build tools export Linux PC, Windows PC and WebGL with the same configured scenes. A successful export is distinct from a tested release. Native Linux, browser and Windows/Wine results must be labelled according to actual execution; Wine never establishes native Windows hardware compatibility.

WebGL needs an HTTP server and a browser with WebGL support; do not open index.html directly as a file. First launch downloads/imports a substantial city asset set, so weak-device memory and load time require testing. No Android/phone claim is made. Record the actual GPU/browser, sound behaviour and input access. A second tested supported platform is required unless the lecturer approves and records an exception.

The final package should be HAI311_FinalProject_6.zip and include report, source with Assets/Packages/ProjectSettings, evidence, presentation, group details and clearly located playable releases. Library/Temp caches and recovery scenes are excluded from the source package. Verify a fresh extraction and import on another machine. Any dependency licensing restriction remains a source-distribution issue to resolve, not something removed by renaming the ZIP.

Ownership ethics and references

The code, integration, tests and documents were prepared with OpenAI Codex assistance at Jayden's request. Each member must review, understand and record actual work. A suggested responsibility is not an authorship claim. Imported city, vehicle, character, road, font and controller assets must retain their original notices. Read the current asset inventory and licence checklist before source redistribution.

The RCC Lite licence present in the project restricts redistribution of the asset itself. Obtain the asset owner's/instructor's clarification for a source hand-in, or use a legitimately distributable dependency arrangement. This report does not declare permissions that have not been established. Public source publication is not implied by creating a local review package.

Design references consulted during this preparation: Schell Games' official overview of The Art of Game Design describes using different design lenses; our stage design uses that as a prompt to examine challenge and

player experience, not as proof of reading a specific textbook edition. Eric Zimmerman's author essay on meaningful play motivates clear action/outcome feedback: stopping leads to visible boarding and confirmed fare credit. The team must consult the actual assigned editions and connect their reading to the design; no invented textbook page citations are included.

Implementation reference: Unity's RaycastNonAlloc documentation explains reusing a result buffer and interpreting its returned hit count. This informs the traffic optimisation and its saturation fallback. Local vendor documentation is used to configure RCC through its public API.

References accessed 8 November 2026:
- Unity, Physics.RaycastNonAlloc scripting reference: https://docs.unity.com/en-us/engine/6000.3/script-reference/unityengine/physics/raycastnonalloc
- Schell Games, The Art of Game Design official overview: https://schellgames.com/art-of-game-design
- Eric Zimmerman, How I Teach Game Design: Lesson 2, Broken games and meaningful play (2014): https://ericzimmerman.wordpress.com/2014/08/11/how-i-teach-game-design-lesson-2-broken-games-and-meaningful-play/ (author commentary; consult the assigned Rules of Play edition separately).
- Supplied HAI311 Final Project & Assessment Guideline, three pages, deadline 13 November 2026.
- Included RCC, EasyRoads, character/art/font notices and project asset register.

The prescribed list also names Hardman, Geig, Thorn, Felicia and Buttfield-Addison/Manning/Nugent. Actual editions/sections consulted by members remain to be recorded in COURSE_READING_REVIEW.csv; this draft does not claim they were read.

Reflection and remaining team work

Technical preparation exposed a mismatch between older documents and the current playable project. We resolved the software scope around one current city route with two deliberate rule configurations, explicit events and a visible ending. Fixes address full-cabin navigation, rivals with inactive cruise/depart behaviour, premature ending display and stale batch smoke behaviour. These are technical findings, not fabricated tester complaints.

The remaining work that requires people or outside permission is authentic tester feedback/fixes/retests, verified member contributions/understanding, course-book consultation, target-machine confirmation and asset source-redistribution clarification. Use the readiness checklist and record real evidence as it arrives. Neither this report nor the slides should be presented as proof that those activities have already happened.

Generated evidence snapshot

integration-Morning.json: 17 passed, 0 failed. Staged automated integration; vehicle positioned at stops; rivals isolated during boarding; not human driving/playtesting

integration-RushHour.json: 17 passed, 0 failed. Staged automated integration; vehicle positioned at stops; rivals isolated during boarding; not human driving/playtesting

traffic-query-results.json: {"vehicles": 10, "samplesPerMode": 120, "iterationsPerSample": 20, "baselineTotalBytes": 491520, "bufferedTotalBytes": 0, "reductionPercent": 100.0, "identicalHitCounts": true, "scope": "Editor CPU physics-query fixture; Mono used-heap delta after forced collection per sample; not whole-game FPS or human playtesting", "utc": "2026-11-08T12:25:48.3947610Z"}

release-Morning.json: 17 passed, 0 failed. Staged automated integration; vehicle positioned at stops; rivals isolated during boarding; not human driving/playtesting

release-RushHour.json: 17 passed, 0 failed. Staged automated integration; vehicle positioned at stops; rivals isolated during boarding; not human driving/playtesting

release-final-Morning.json: 17 passed, 0 failed. Staged automated integration; vehicle positioned at stops; rivals isolated during boarding; not human driving/playtesting

web-integration-RushHour.json: 17 passed, 0 failed. Staged automated integration; vehicle positioned at stops; rivals isolated during boarding; not human driving/playtesting

web-integration-final-Morning.json: 17 passed, 0 failed. Staged automated integration; vehicle positioned at stops; rivals isolated during boarding; not human driving/playtesting

windows-wine-Morning.json: 17 passed, 0 failed. Staged automated integration; vehicle positioned at stops; rivals isolated during boarding; not human driving/playtesting

windows-wine-RushHour.json: 17 passed, 0 failed. Staged automated integration; vehicle positioned at stops; rivals isolated during boarding; not human driving/playtesting

A generated report is not final group sign-off. Complete the genuine human evidence records and asset permissions review before representing submission readiness.